Contents

1 - Upgradeability 2
2 - Registry launch (settings protocol) 2
3 - Registry change — Propose (settings protocol) 3
4 - Registry change — Apply (upgrade, settings protocol) 4
5 - Forced genesis (state token mint) 5
6 - Forced spend (beacon indirection) 6
7 - Opt-in native action (Tick) 7
8 - Opt-in upgrade (migration) 8

1 - Upgradeability

Cardano validators are immutable and content-addressed: a validator's address is its code hash, so its logic can never change in place. This protocol provides the two eUTxO-native ways to evolve a deployed protocol, both built on the **Config / Parameter Management** primitive:

- **Migration** (opt-in) — deploy new immutable code and let each user move **their own** UTxO to it.
- **Beacon indirection** (forced) — a thin forwarding validator delegates its logic to a script hash held in a governed **registry**, so the logic can be repointed without moving funds.

Both share a single registry `R`, stored as the settings UTxO's `current` value:

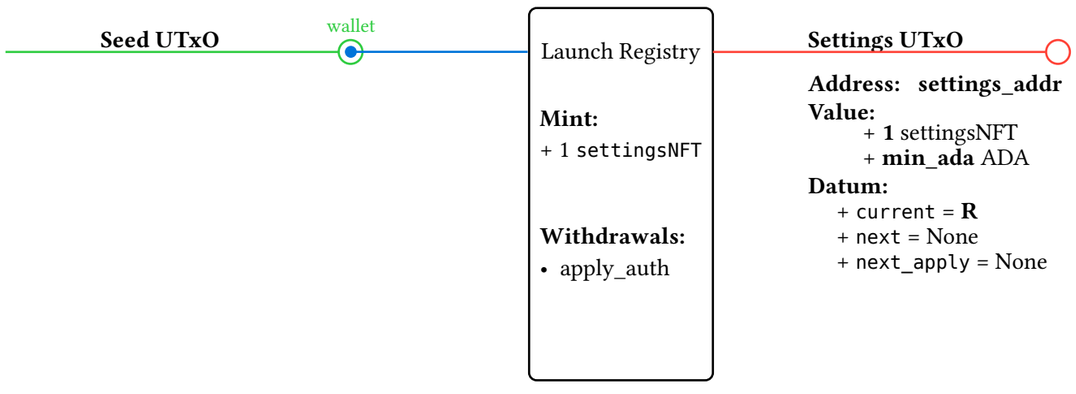
$$R = \{h_{\text{proto}} : h_{\text{delegate}}, \ldots\}$$

where each key `h_proto` is the immutable hash of a protocol validator, and its value `h_delegate` is the script that currently governs it. Registry changes reuse the settings protocol's `Propose` / `Apply` split, its `apply_delay` timelock, and its pluggable `propose_auth` / `apply_auth` authorization.

The registry is a settings value: launching it, proposing a change, and applying a change are the **settings** protocol's own transactions (20), reproduced here for completeness — see `../settings/design.typ` for the full lifecycle. Only the value is registry-shaped. The mechanism-specific transactions follow.

2 - Registry launch (settings protocol)

Creates the settings UTxO that holds the registry. One instance governs any number of protocols.

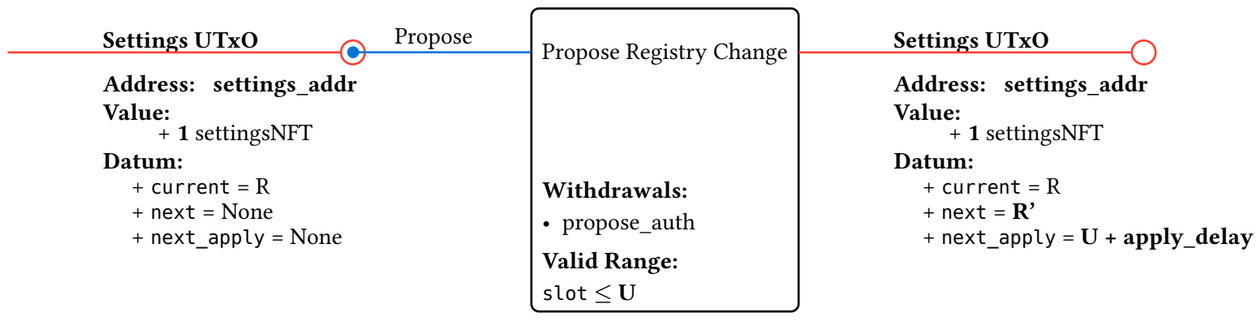


Notes:
- `R` is the registry: a map from a protocol's own hash to its active delegate script hash.
- `settingsNFT` uniquely identifies the instance; minting requires spending the one-shot `seed_utxo`, so at most one instance exists.
- The reference settings validator accepts any value; consumers reject a malformed registry at spend time (`decode_registry`).
- `Close` burns the one-shot NFT and the instance can never be relaunched, so closing permanently bricks every registered consumer: forced forwarders become unspendable and opt-in V1 UTxOs lose their value exit. A production registry should use a settings fork without `Close`.

Figure 1: Launch registry instance

3 - Registry change — Propose (settings protocol)

Stages a new registry behind the apply delay. A new proposal supersedes any pending one.

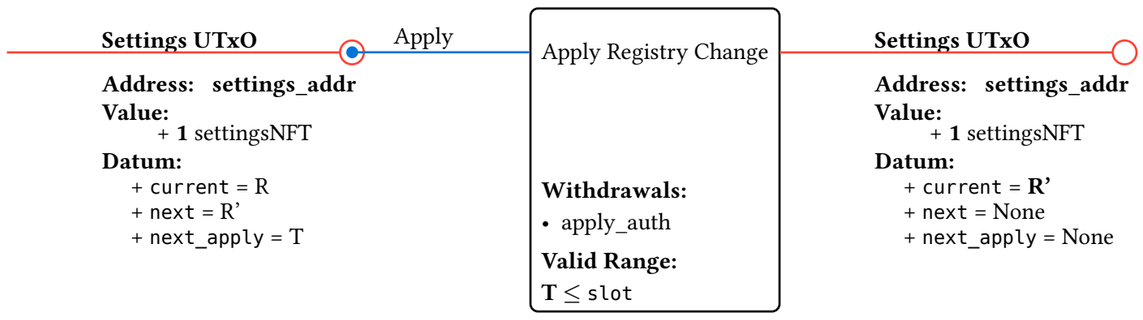


Notes:
- `R' != R` and `validate_datum(R')`.
- The proposal can only change the registry; it cannot be applied yet.
- The delay is anchored on the validity upper bound `U`, which a proposer cannot backdate: the transaction cannot land after `U`.

Figure 2: Propose registry change

4 - Registry change — Apply (upgrade, settings protocol)

Commits a matured proposal after the timelock. This is the only transaction that changes a protocol's behaviour.

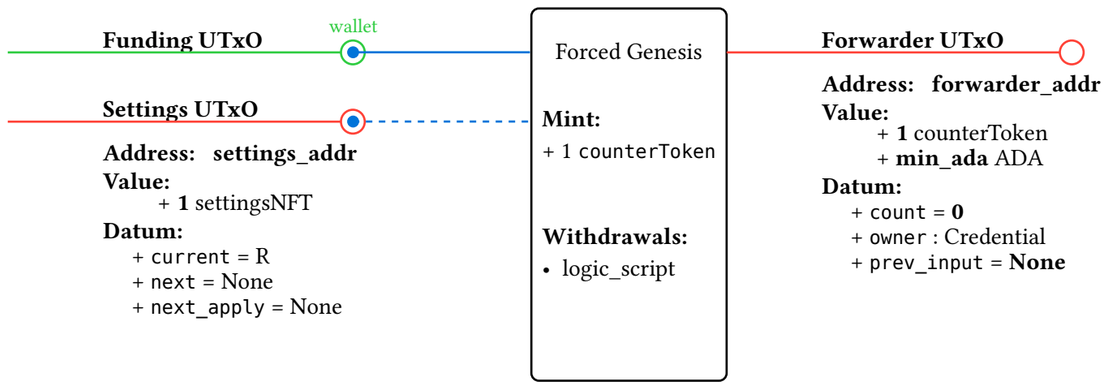


Notes:
- Repointing `R[h_proto] = h_new` **is** the upgrade. For **forced**, `h_new` is the new logic script — funds never move and the change applies to everyone at once. For **opt-in**, `h_new` is the new migration script — each user then chooses to migrate.
- Timelocked: valid only when `now >= T`, and `T` is at least `apply_delay` after the proposal landed, even against malicious governance. The delay gives users a window to inspect the new delegate and, if the current logic has an exit path, to leave (spec §6.a). For opt-in, it is the window to migrate.

Figure 3: Apply registry change

5 - Forced genesis (state token mint)

The forwarder is also the state token's minting policy. A mint forwards like a spend, and the logic admits exactly one token paid to a fresh counter.

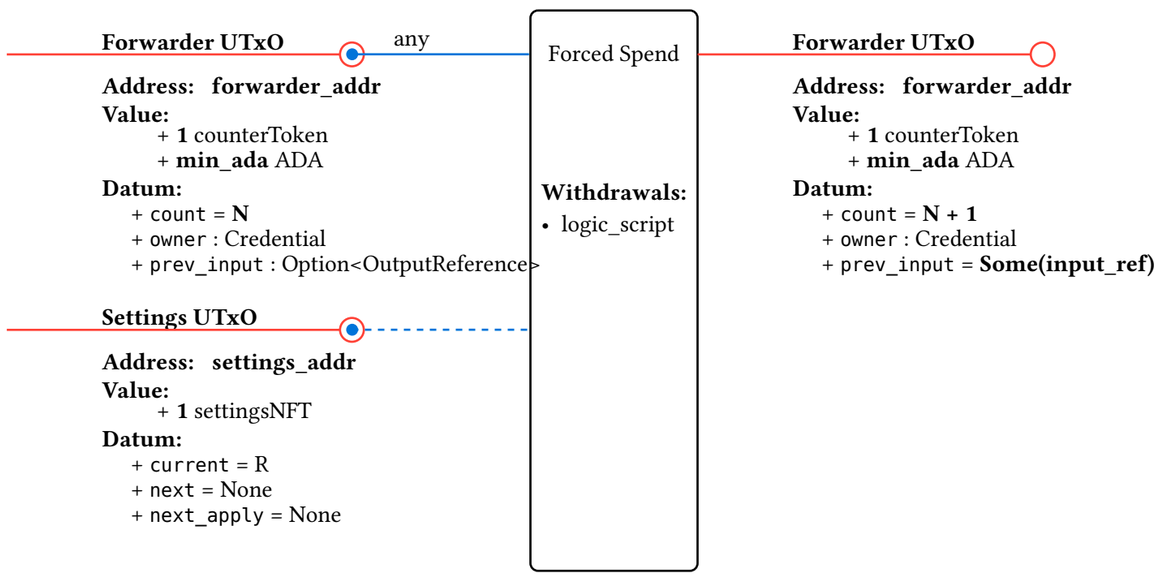


Notes:
- The forwarder runs for the mint (`own_hash` is the policy id) and only checks that the registered logic runs.
- The logic admits a mint under the forwarder's policy only as one `counterToken` paid, with no reference script, to a fresh counter at the forwarder's own address. Burns and other names are rejected.
- After a schema upgrade, genesis writes the new datum shape (spec §4.b).

Figure 4: Forced genesis transaction

6 - Forced spend (beacon indirection)

A thin forwarder holds state but carries no logic. Every use of it, for any purpose, must be accompanied by the logic script registered for the forwarder's own hash. The logic validates every item delegated to it.



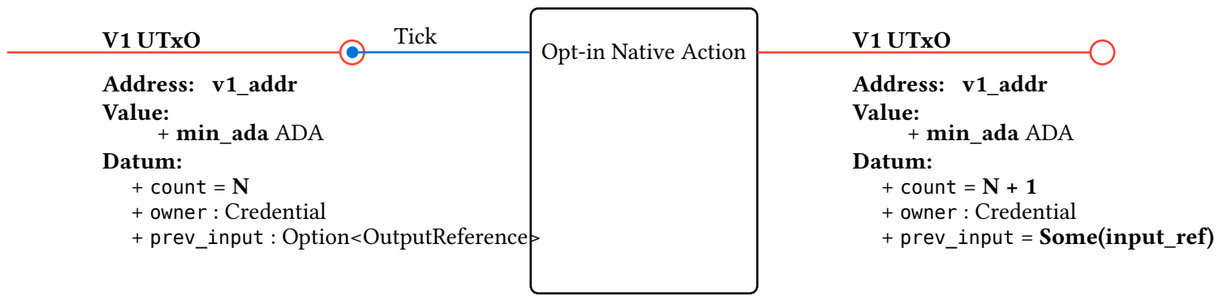
Notes:

- `validate_forward`: `own_hash` is the forwarder's own hash (here the spent input's payment credential), `R[own_hash] = logic_script`, and `logic_script` must be present as a withdrawal (withdraw-0 convention).
- The forwarder's redeemer is opaque: the **logic script** is responsible for validating **every** item delegated to it (inputs, mints, withdrawals, certificates, votes), not just one, and rejects kinds it does not handle.
- `counterToken` is the state token (`forwarder_hash.counter`): the logic ticks only inputs that carry it, so a UTxO paid straight to the forwarder address is not a counter.
- Each continuation names the spent UTxO in its `prev_input` tag, so one output can only settle the single input it names — several UTxOs can be forwarded in one transaction without an N⟶1 collapse.
- The continuation preserves the input's non-ADA assets exactly, holds at least its lovelace, and carries no reference script.
- A registry repoint changes behaviour for all users at once: this is a **forced** upgrade, so timelock and bounded governance are mandatory.
- A repoint does not touch the datum. A logic that changes the schema reads every shape ever written and writes the new one on each continuation (convert on touch, spec §4.b).

Figure 5: Forced spend transaction

7 - Opt-in native action (Tick)

An opt-in protocol keeps its native logic for ordinary actions; only the `Upgrade` path forwards.

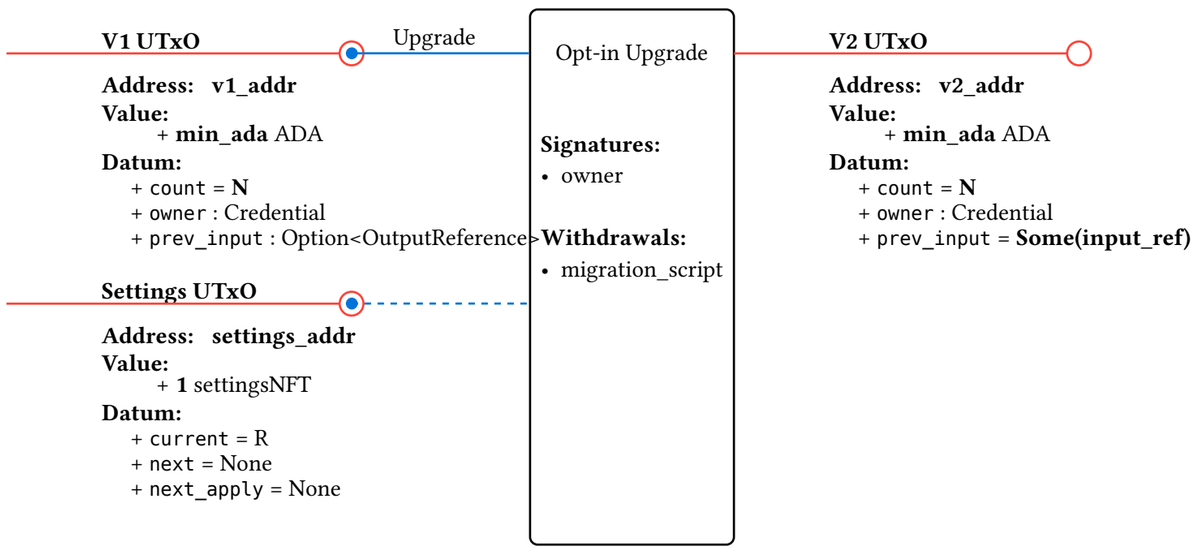


Notes:
- V1's own validator enforces the action natively; the registry is not consulted on this path.
- The continuation is tagged with the spent UTxO (`prev_input`), preserves its non-ADA assets exactly, does not decrease its lovelace, and carries no reference script.

Figure 6: Opt-in native action

8 - Opt-in upgrade (migration)

Moves a user's state to a new immutable version. Consent is enforced natively in V1; the registered migration script validates the transformation.



Notes:
- `validate_upgrade`: `R[v1_addr] = migration_script` must be present **and** `owner` must authorize (consent enforced natively in V1).
- The migration script checks the V1 input was spent with `Upgrade` and that its state reappears at `v2_addr` tagged `prev_input = input_ref`, with `count`, `owner` and the non-ADA assets preserved and no reference script.
- A governance repoint of `migration_script` changes **how** a migration is performed but can never move funds without `owner`'s signature — this is what makes the mechanism **opt-in**.

Figure 7: Opt-in upgrade transaction